

CAHIER DES CHARGES

Application de gestion locative saisonnière — Société Neige & Soleil

BTS SIO SLAM — Épreuve E6 — Session 2026

Projet	Application web PHP/MySQL (client léger) + Application bureau Java/Swing (client lourd)
Société	Neige & Soleil (N & S) — Location saisonnière — Vallée du Queyras, Briançon
Auteure	CHENINE Ayat — BTS SIO SLAM — École IRIS Paris
Période	Septembre 2025 — Mai 2026
Version	1.0 — Juin 2026
Épreuve	BTS SIO SLAM — Épreuve E6 — Coefficient 4

Résumé : Ce cahier des charges décrit les spécifications fonctionnelles et techniques du projet Neige & Soleil développé dans le cadre du BTS SIO SLAM épreuve E6. Deux applications complémentaires ont été réalisées : une application web PHP/MySQL (client léger, architecture MVC) et une application bureau Java/Swing (client lourd, architecture MVC + DAO JDBC), permettant la gestion complète des propriétaires, gîtes, clients et réservations saisonnières de la société Neige & Soleil.

I. Présentation générale du projet

A. Contexte et présentation de l'entreprise

La société **Neige & Soleil (N & S)** exerce son activité dans la vallée du Queyras, près de Briançon. Elle est spécialisée dans la location d'appartements de montagne à la saison et la location de matériel (ski, randonnée).

Élément	Détail
Activité principale	Location d'appartements à la montagne (saisonnier)
Activité secondaire	Location de matériel de montagne (ski, randonnée)
Effectif	1 directeur · 1 secrétaire-comptable · 2 commerciaux · 1 ouvrier
Localisation	Vallée du Queyras — Briançon (Hautes-Alpes, France)

B. Origine et justification du projet

Neige & Soleil gère ses données sur des fichiers papier et tableurs Excel. Cette organisation rendait le suivi long et sujet aux erreurs. Le projet vise à moderniser les outils de gestion avec deux applications complémentaires : un site web accessible depuis un navigateur et une application bureau installée sur le poste administrateur.

C. Périmètre — deux réalisations complémentaires

- Réalisation 1 — Client léger :** Application web PHP/MySQL accessible depuis un navigateur. Architecture MVC côté serveur (index.php → Contrôleur → Modèle PDO → Vues PHP). Déployée sur Apache via XAMPP.
- Réalisation 2 — Client lourd :** Application bureau Java/Swing installée sur le poste administrateur. Architecture MVC + DAO. Connexion directe à MySQL via JDBC (PreparedStatement). Développée sous Eclipse.

II. Acteurs et besoins

A. Acteurs du système

Acteur	Statut BDD	Accès et rôle
Administrateur	statut = admin	Accès complet aux 4 tables — tableau de bord — statistiques
Propriétaire privé	statut = privé	Gestion de ses gîtes — consultation de ses réservations
Propriétaire public	statut = public	Accès identique au privé — visible dans le catalogue

Acteur	Statut BDD	Accès et rôle
Client / Locataire	table client	Ses réservations uniquement — ajout de réservation — pas d'accès aux données tierces

B. Besoins fonctionnels

Propriétaires — CRUD complet avec trois niveaux de statut (privé / public / admin). Authentification sécurisée. Un propriétaire peut posséder plusieurs gîtes.

Gîtes — CRUD complet : adresse, surface, nombre de pièces, loyer journalier. Association obligatoire à un propriétaire. Suppression en CASCADE.

Clients — CRUD complet. Authentification sécurisée. Un client connecté ne voit que ses propres réservations. Suppression en CASCADE.

Réservations — CRUD complet avec dates, prix, transport (voiture/train/avion), assurance, statut (en cours/terminé/non réservé) et rapport. Filtrage par statut.

Tableau de bord (admin uniquement) — Compteurs temps réel, chiffre d'affaires sur réservations terminées, liste des dernières réservations.

III. Architecture technique

A. Comparatif Client Léger vs Client Lourd

Critère	Client Léger — Réalisation 1	Client Lourd — Réalisation 2
Technologies	PHP 8, HTML5, CSS3, JavaScript, SQL	Java, Swing, JDBC, MySQL
Serveur	Apache via XAMPP	JVM locale installée sur le poste
Accès BDD	Via PHP/PDO côté serveur Apache	Connexion directe JDBC → MySQL
Interface	Navigateur web (HTTP, HTML généré)	Fenêtres Java (JFrame/JPanel/JTabbedPane)
Architecture	MVC côté serveur (index.php front controller)	MVC + DAO (Data Access Object)
Sécurité SQL	Requêtes PDO préparées	PreparedStatement anti-injection
Déploiement	Aucune installation côté client	Installation JDK + jar sur le poste
IDE utilisé	VS Code	Eclipse IDE 2026

B. Architecture MVC — Client Léger (PHP)

Point d'entrée unique **index.php** (front controller) → **controleur.class.php** (routeur + droits + sessions) → **modele.class.php** (toutes les requêtes PDO préparées) → **vues PHP** (génèrent le HTML). La connexion est gérée par un Singleton PDO dans **config_db.php**. Le navigateur ne reçoit que du HTML — aucune logique côté client.

C. Architecture MVC — Client Lourd (Java)

Package **controleur** : NeigeEtSoleil.java (Main) + Controleur.java (logique métier). Package **modele** : bdd.java (connexion JDBC) + modele.java (tout le SQL en PreparedStatement) + 4 classes POJO (Proprietaire, Client, Gite, Reservation). Package **vue** : VueConnexion, VuePrincipale et 6 panels CRUD (PanelGenerique comme base réutilisable, PanelDashboard, PanelProprietaires, PanelClients, PanelGites, PanelReservations).

IV. Besoins non fonctionnels

Exigence	Description — implémentation concrète dans le projet
Sécurité SQL	PDO préparées (PHP) + PreparedStatement (Java) — aucune concaténation de chaînes dans les requêtes
Sécurité auth.	Sessions PHP avec vérification des droits à chaque requête — gestion des rôles (admin/privé/public/client)
Intégrité BDD	Clés étrangères InnoDB avec ON DELETE CASCADE — confirmation avant toute suppression
Interface	Navigation par onglets (Java) — formulaires avec messages d'erreur clairs — recherche sur toutes les listes

Exigence	Description — implémentation concrète dans le projet
Compatibilité	Client léger : navigateur moderne (Chrome, Firefox) — Client lourd : Windows 10/11 + JDK installé
Évolutivité	Architecture MVC dans les deux applications — code commenté — ajouts prévus : export CSV, notifications

V. Domaine de gestion — Règles métier

A. Règles sur les réservations

Étape	Règle métier implémentée
Création	Statut initial automatique : "en cours" — saisie obligatoire des dates, prix, transport
Arrhes	20 % du montant total à verser sous 8 jours — confirmation par reçu
Solde	Exigible 3 semaines avant le début de la location
Annulation	Automatique si arrhes ou solde non reçus dans les délais — arrhes perdues
Arrivée	Chèque de caution + état des lieux signé
Départ	Restitution caution si aucun problème — procédure litige si désaccord
Suppression	CASCADE automatique : supprimer un client ou gîte supprime ses réservations associées

B. Règles sur les contrats locatifs

- Un contrat = un appartement (un propriétaire peut avoir plusieurs gîtes)
- Durée : 1 an (1er octobre → 30 septembre) — Conservation des inactifs : 5 ans minimum
- Relances automatiques en août — Radiations possibles sur demande du propriétaire
- Saison de location : de la 2e semaine de décembre à la dernière semaine de septembre

VI. Critères de validation et tests

A. Tableau de tests fonctionnels

Test	Résultat attendu	Statut
Connexion Admin / Proprio / Client	Accès selon le rôle	■
CRUD Propriétaires	Ajout, modif, suppression, liste OK	■
CRUD Gîtes avec liaison proprio	Mise à jour BDD correcte	■
CRUD Clients	Suppression CASCADE des réservations	■
CRUD Réservations	Contrôle champs + enregistrement	■
Tableau de bord	Compteurs et CA corrects	■
Sécurité SQL	PreparedStatement — aucune injection possible	■

B. Critères de réception

- CRUD complet et fonctionnel sur les 4 tables (proprio, client, gîte, reservation)
- Authentification sécurisée avec gestion des rôles opérationnelle
- Aucune donnée accessible sans authentification préalable
- Architecture MVC respectée dans les deux applications
- Requêtes SQL 100% préparées (PDO et PreparedStatement) — zéro injection possible
- Code versionné sur GitHub avec historique de commits